

```
In [1]: import pandas as pd

df = pd.read_csv("water_potability.csv")
df.head()
```

```
Out[1]:
```

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_c
0	NaN	204.890455	20791.318981	7.300212	368.516441	564.308654	10.
1	3.716080	129.422921	18630.057858	6.635246	NaN	592.885359	15.
2	8.099124	224.236259	19909.541732	9.275884	NaN	418.606213	16.
3	8.316766	214.373394	22018.417441	8.059332	356.886136	363.266516	18.
4	9.092223	181.101509	17978.986339	6.546600	310.135738	398.410813	11.

```
In [2]: df.shape
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3276 entries, 0 to 3275
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ph                    2785 non-null   float64
1   Hardness              3276 non-null   float64
2   Solids                3276 non-null   float64
3   Chloramines           3276 non-null   float64
4   Sulfate               2495 non-null   float64
5   Conductivity          3276 non-null   float64
6   Organic_carbon        3276 non-null   float64
7   Trihalomethanes       3114 non-null   float64
8   Turbidity             3276 non-null   float64
9   Potability            3276 non-null   int64
dtypes: float64(9), int64(1)
memory usage: 256.1 KB
```

```
Out[2]:
```

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity
count	2785.000000	3276.000000	3276.000000	3276.000000	2495.000000	3276.000000
mean	7.080795	196.369496	22014.092526	7.122277	333.775777	426.205111
std	1.594320	32.879761	8768.570828	1.583085	41.416840	80.824064
min	0.000000	47.432000	320.942611	0.352000	129.000000	181.483754
25%	6.093092	176.850538	15666.690297	6.127421	307.699498	365.734414
50%	7.036752	196.967627	20927.833607	7.130299	333.073546	421.884968
75%	8.062066	216.667456	27332.762127	8.114887	359.950170	481.792304
max	14.000000	323.124000	61227.196008	13.127000	481.030642	753.342620

```
In [3]: df.isnull().sum()
```

```
Out[3]: ph                491
Hardness                0
Solids                  0
Chloramines             0
Sulfate                 781
Conductivity            0
Organic_carbon          0
Trihalomethanes        162
Turbidity               0
Potability              0
dtype: int64
```

```
In [4]: df['Potability'].value_counts()
```

```
Out[4]: Potability
0      1998
1      1278
Name: count, dtype: int64
```

```
In [5]: df.fillna(df.median(), inplace=True)
```

```
In [6]: df.isnull().sum()
```

```
Out[6]: ph                0
Hardness                0
Solids                  0
Chloramines             0
Sulfate                 0
Conductivity            0
Organic_carbon          0
Trihalomethanes         0
Turbidity               0
Potability              0
dtype: int64
```

```
In [7]: X = df.drop('Potability', axis=1)
y = df['Potability']
```

```
In [8]: from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
In [9]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

```
In [10]: from sklearn.linear_model import LogisticRegression

lr = LogisticRegression()
lr.fit(X_train, y_train)
```

Out[10]:

▼ **LogisticRegression** ⓘ ?
 LogisticRegression()

In [11]: `from sklearn.tree import DecisionTreeClassifier`

```
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
```

Out[11]:

▼ **DecisionTreeClassifier** ⓘ ?
 DecisionTreeClassifier(random_state=42)

In [12]: `from sklearn.ensemble import RandomForestClassifier`

```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

Out[12]:

▼ **RandomForestClassifier** ⓘ ?
 RandomForestClassifier(random_state=42)

In [13]: `from sklearn.svm import SVC`

```
svm = SVC(probability=True)
svm.fit(X_train, y_train)
```

Out[13]:

▼ **SVC** ⓘ ?
 SVC(probability=True)

In [14]: `y_pred = rf.predict(X_test)`In [15]: `from sklearn.metrics import classification_report, confusion_matrix, accuracy_score`

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Accuracy: 0.6737804878048781

[[350 62]

[152 92]]

	precision	recall	f1-score	support
0	0.70	0.85	0.77	412
1	0.60	0.38	0.46	244
accuracy			0.67	656
macro avg	0.65	0.61	0.61	656
weighted avg	0.66	0.67	0.65	656

In [16]: `from sklearn.metrics import roc_auc_score`

```
y_prob = rf.predict_proba(X_test)[:,-1]  
roc_auc_score(y_test, y_prob)
```

Out[16]: 0.6871617857711284

In []: